Selfish Deep Q-Learners Can Learn to Hunt the Stag

Autonomous and Adaptive Systems | project

Marco Zocco Ramazzo
Master's Degree in Artificial Intelligence, University of Bologna
marco.zoccoramazzo@studio.unibo.it

Abstract

This project studies when two independently learning DQN agents can cooperate in the *Hunt* game, a grid-world Stag Hunt: hunting the stag together pays most, but an agent that hunts alone is punished. The starting point is the environment's default settings. There, plain DQN agents learn to forage instead (1 of 12 runs hunts), even when they share rewards: the obstacle is coordination during learning, not incentives. Changing only the learning rule fixes it: with hysteretic updates, which down-weight negative TD errors, or NoisyNet exploration resampled once per episode, selfish agents hunt in 11/12 and 12/12 runs, with shared or fully separate networks. Building on this, we test the learned cooperation beyond the defaults. It survives punishments from -1 to -20 and grids up to 9×9, and weakens only when foraging becomes nearly as rewarding as hunting. It is passive (the agents meet and let the stag walk into them) and partner-specific: paired with agents from other runs, cooperative agents hunt in only 44% of the pairs. When the stag moves randomly, the agents learn to chase it, but only once exploration has ended and after about 0.5M (shared network) to 2M (separate networks) training steps.

1 Introduction

In a Stag Hunt, two players can cooperate on a risky but rewarding goal (the stag) or each take a small, safe reward (a plant). Both are equilibria, and learning agents tend to end up in the safe one: early in training each agent learns from a partner who is still exploring, so going for the stag is usually punished, and it looks worse than foraging even though it is the best choice when both commit.

This project studies cooperation between two learning agents in the *Hunt* game of the Gym-Stag-Hunt environments [3] (Figure 1), in four steps:

1. **Cooperation at the default settings.** As a starting point, can two selfish agents learn to hunt with the environment's default rewards, without reshaping the game or making them care about each other? Plain DQN fails (Section 3.1); two learning rules, hysteretic DQN [2, 4] and NoisyNet exploration [1] resampled per episode, succeed reliably (Section 3.2). Section 3.3 describes what the agents learn.
2. **Unseen partners.** Do agents from different runs cooperate with each other (Section 3.4)?
3. **Rewards and state size.** Under which punishments and temptations is cooperation still possible, and does it scale to larger grids (Section 3.5)?
4. **A randomly moving stag.** By default the stag walks towards the agents. Can they learn to hunt actively when it moves randomly, and what limits this (Section 3.6)?

Project report, Autonomous and Adaptive Systems 2025/26, University of Bologna.

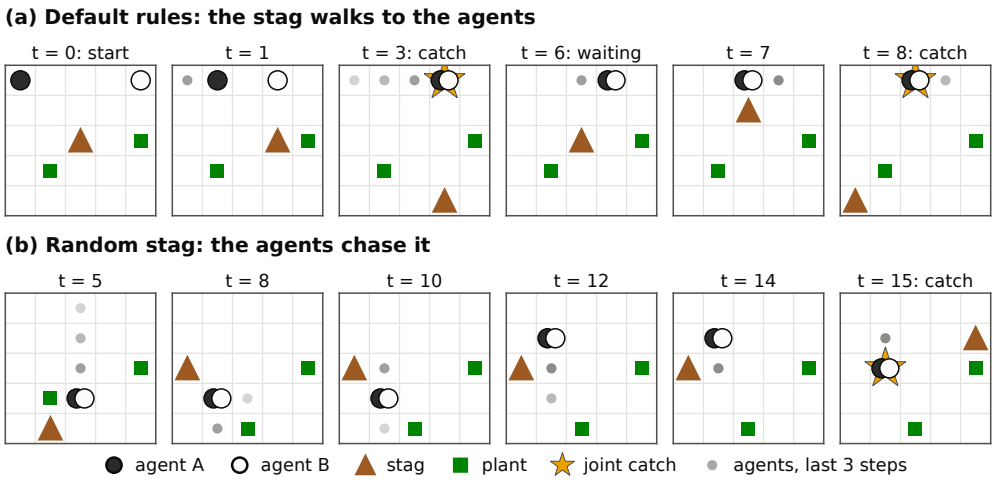


Figure 1: The Hunt game, played by two trained agents (greedy policies, shared network, hysteretic DQN). Agents start in the top corners; a joint catch pays $+5$ to each agent, an agent alone on the stag is mauled (-5), a plant pays $+1$. After a catch the stag reappears at a random cell. **(a)** Default rules, where the stag walks towards the nearest agent: the agents meet on the top row and wait for it. **(b)** Randomly moving stag (3M training steps): the agents move as one unit and follow the stag until it steps onto their cell.

Table 1: The five agents. All share the architecture and hyperparameters given in the text.

Agent	Update rule	Exploration
Plain DQN	TD loss	ε: $1 \to 0.05$ in the first half of training
DQN, $\varepsilon_0 = 0.1$ (control)	TD loss	ε: $0.1 \to 0.05$ in the first half
Hysteretic DQN	$\beta = 0.2$ on $\delta < 0$ (Eq. 1)	as plain DQN
NoisyNet, per step	TD loss	noisy weights ($\sigma_0 = 0.5$), new noise every step
NoisyNet, per episode	TD loss	noisy weights ($\sigma_0 = 0.5$), new noise every episode

2 Method

Environment. Figure 1 shows the game: a 5×5 grid with two agents, one stag and two plants, and five actions (four moves and stand). By default the stag moves one cell towards the nearest agent at every step, and it stays where it is after mauling an agent. Each agent observes the (x, y) positions of all entities, scaled to $[0, 1]$, with its own position first. All rewards and dynamics are left at their defaults; the only change is that episodes are truncated after 100 steps instead of 1,000, to see more starting layouts in the same training budget. Since the game has no terminal states and truncation is not treated as termination, this does not change the task.

Agents. All agents are DQN learners with Double DQN targets [6] and share the same hyperparameters: an MLP with two hidden layers of 64 units, Huber loss, Adam (learning rate $5\cdot10^{-4}$), $\gamma = 0.95$, a replay buffer of 50k transitions, minibatches of 64, one update per environment step after the first 1,000 steps, a target network synced every 1,000 steps, and gradient-norm clipping at 10. Unless stated otherwise, training lasts 300k environment steps. Each agent acts on its own observation only, and can be trained with *separate* networks (two fully independent learners, each with its own replay buffer) or with a *shared* network (one learner trained on both agents' transitions, with a one-hot agent ID appended to the input, because the game is not symmetric: when both agents are equally close, the stag goes after agent B). Agent i learns from the prosocial reward $\tilde{r}_i = (1-\alpha)\, r_i + \alpha\, r_j$ [5], with $\alpha = 0$ (selfish) or $\alpha = 0.5$ (fully shared reward); all reported returns use the true environment rewards. The five agents differ only in their update rule and exploration (Table 1).

Learning rules. *Hysteretic DQN* [2, 4] weights the per-sample loss by $\beta < 1$ when the TD error $\delta = y - Q(s, a)$ is negative:

$$\mathcal{L} = \mathbb{E}\big[\, w(\delta)\, \mathrm{Huber}(\delta)\,\big], \qquad w(\delta) = \begin{cases} \beta & \delta < 0 \\ 1 & \delta \geq 0 \end{cases}, \qquad y = \tilde{r} + \gamma\, Q_{\bar{\theta}}\big(s', \arg\max_{a'} Q_\theta(s', a')\big), \tag{1}$$

Table 2: Plain DQN with a shared network (3 seeds, 300k steps): hunting runs and catches per 100 steps.

	mauling penalty -1	mauling penalty -5 (default)
selfish ($\alpha = 0$)	0/3 (0.5 ± 0.1)	0/3 (0.0 ± 0.0)
shared reward ($\alpha = 0.5$)	3/3 (26.7 ± 4.0)	0/3 (0.0 ± 0.0)

which for small errors equals a learning rate β times smaller on bad surprises: an agent forgets a good joint outcome more slowly than it learns it, and is less discouraged by failures caused by its partner's exploration; $\beta = 1$ recovers DQN.

NoisyNet [1] replaces ε-greedy with learned factorised Gaussian noise on all weights. The original method resamples the noise every step; we also test resampling it once per episode, which makes each agent follow one consistent perturbed policy for a whole episode. Since NoisyNet explores less than ε-greedy at the start of training, the control agent (plain DQN with ε starting at 0.1) checks whether this alone explains its results.

Baselines. To put the learned agents in context, scripted pairs of agents are evaluated with the same protocol (`baselines.py`). Two *random* agents give a lower bound: they make 1.1 catches per 100 steps, but are mauled 35 times. Two agents that always walk towards the stag (*always hunt*) make only 1.8 catches, because they rarely arrive together, and two agents that always walk to the nearest plant (*always forage*) earn 34–40 per agent. As an upper reference, a *joint planner* controls both agents at once: it knows the stag's rule and, at every step, picks the pair of actions with the best expected immediate reward, with a small bonus for staying close to the stag. It makes 51.2 catches per 100 steps at the default settings. The planner is not a learning agent and not optimal (it only looks one step ahead); it marks what coordinated play can achieve.

Evaluation. After training (300k environment steps unless stated otherwise), the greedy policy is evaluated on 200 episodes of 100 steps with fixed layouts (seeds 0–199, disjoint from the training seeds). The main metric is the number of joint catches per 100 steps. A run *hunts* if it makes more than 8 catches: 8 catches are worth 40 per agent, about the return of an agent that only forages. Outcomes are bimodal (a run either hunts or forages), so we report the number of hunting runs, and use Fisher's exact test to compare it between methods. In total 267 runs were trained, about 15 CPU-minutes each for 300k steps.

3 Experiments and results

3.1 Plain DQN does not discover the joint catch

Table 2 shows our starting point. Plain DQN learns to hunt only when both conditions hold: the agents share rewards and the punishment is mild. At the default penalty, even fully prosocial agents forage. This is not because hunting is unprofitable: the planner earns about 258 per agent per 100 steps at the default rewards, against about 40 for foraging. The agents never *discover* it: no mid-training evaluation of these runs exceeded 0.4 catches. With random actions, an agent that approaches the stag is mauled far more often than both agents meet on it, so "avoid the stag" is learned long before a joint catch occurs. Across all our conditions, plain DQN hunted in only 4 of its 65 runs, three of them in the $\alpha = 0.5$, -1 cell of Table 2.

3.2 Changing the learning rule is enough

Figure 2a compares the learning rules at the default rewards. Pooling the four settings (shared or separate networks, $\alpha \in \{0, 0.5\}$), plain DQN hunts in 1/12 runs. Hysteretic DQN reaches 11/12 ($p < 10^{-3}$, Fisher's exact test against plain DQN), NoisyNet resampled per step 10/12 ($p < 10^{-3}$), and NoisyNet resampled per episode 12/12 ($p < 10^{-4}$). The control with less early exploration hunts in 3/12 runs ($p = 0.59$), so the effect of NoisyNet is not just a smaller ε at the start.

Three further points stand out. First, cooperation does not need shared rewards: with these rules, $\alpha = 0$ and $\alpha = 0.5$ give the same results, so selfish agents are enough. Second, it does not need a shared network: with two fully independent learners, hysteretic DQN hunts in 5/6 runs and NoisyNet

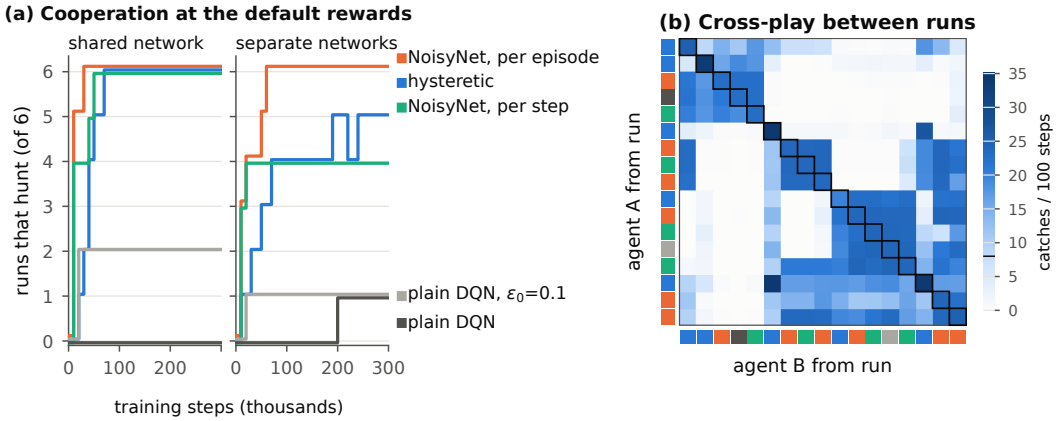


Figure 2: Default environment. **(a)** Runs (out of 6: $\alpha \in \{0, 0.5\} \times 3$ seeds) whose greedy policy hunts (> 8 catches per 100 steps) during training, with a shared network (left) and separate networks (right). Lines that end at the same value are offset slightly. **(b)** Cross-play between the 17 separate-network runs that hunt in self-play: cell (i, j) pairs agent A of run i with agent B of run j (200 episodes); boxed cells are self-play. The coloured strips give each run's learning rule (colours as in (a)); runs are ordered by hierarchical clustering. The line on the colour bar marks the hunting threshold.

per episode in 6/6. Shared networks learn faster and fail less often (6/6 for all three rules). Third, the two rules learn in different ways. Most NoisyNet runs already hunt at the first evaluation (10k steps), while hysteretic runs start after a median of 40k steps, but then reach higher catch rates (33.7 against 25.6 catches on average over hunting runs). The best pair (shared, hysteretic) makes 51.4 catches, on par with the joint planner.

3.3 What the agents learn: meet, then wait

We analysed all 135 hunting runs (over every condition) by replaying their greedy policies for 200 episodes. Every hunting pair first meets: the agents reach the same cell in at least 98% of the episodes, on average within 2–12 steps, and then move as one unit (median co-location 86% of the steps). In a median of 97% of the catches, both agents are already on the same cell when the stag arrives. The agents spend 62% of the steps without moving, but choose the stand action in only 20% of all steps: most of the time they "wait" by walking into a wall. They do not chase the stag: they exploit the default rule that the stag walks towards the nearest agent, and let it walk into them (Figure 1a). This is still genuine cooperation, since a single agent waiting alone is mauled. Being together, however, is not enough: plain DQN with a shared network and $\alpha = 0$ also keeps the agents together for about 60% of the steps, but they forage side by side and never hunt.

3.4 Cooperating with unseen partners

Different runs find the same strategy, but do they find the same *conventions* (cross-play, Figure 2b)? Of the 30 separate-network runs at the default settings, 17 learned to hunt. Each of them, played with its own training partner (self-play), makes 25.2 catches on average. Pairing agent A of one run with agent B of another gives 272 cross-play pairs, of which only 44% hunt (8.8 catches on average), and the agents meet in only 71% of the episodes. A failed pair does worse than two foragers: one agent waits next to the stag, the partner does not come, and since the stag does not leave after a mauling, the waiting agent is mauled again and again (41.9 maulings per 100 steps, averaged over the 272 pairs). The matrix has three blocks of mutually compatible runs. Each block contains runs trained with different learning rules, and two runs trained with the same rule and α but different seeds are no more compatible than two runs trained differently (45.8% against 44.0% of pairs hunt). So the convention a pair agrees on depends on the seed, i.e. on chance during training, not on the method. A few "generalist" runs work with most partners, while the best self-play pair (34.0 catches) works with only a few.

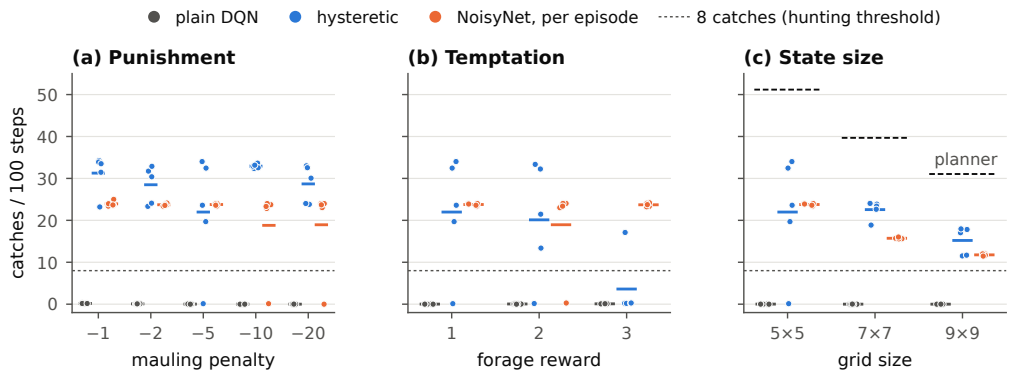


Figure 3: Changing one environment parameter from its default (-5, 1, 5×5). Final catches per 100 steps of each run (dots) and their mean (bars), for separate networks, $\alpha = 0$ and 5 seeds. In (c), dashed lines mark the joint planner.

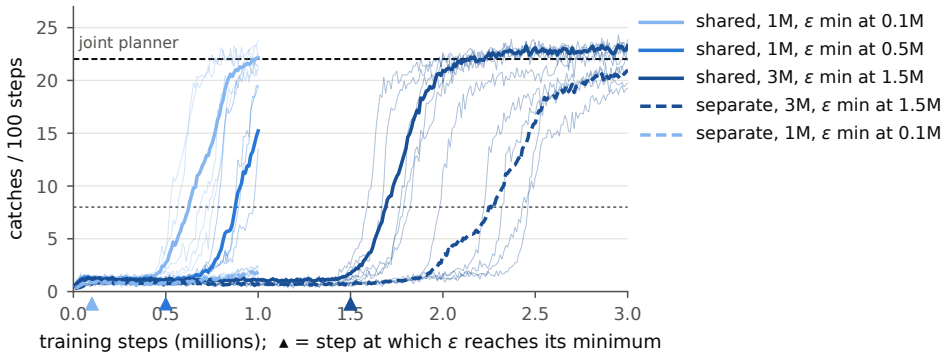


Figure 4: Randomly moving stag, hysteretic DQN: catches per 100 steps during training, mean of 5 seeds (thick) and single seeds (thin), for different training lengths and ε schedules. Triangles mark the step at which ε reaches its minimum; the dashed line marks the joint planner.

3.5 Rewards, temptation, and state size

From here on we compare plain DQN with the two best rules at the default settings, hysteretic DQN and NoisyNet per episode. We varied one parameter at a time, with separate networks, selfish agents and 5 seeds per condition (Figure 3). Plain DQN never hunts, in any condition, even with a mauling penalty of only -1: without shared rewards, a milder punishment is not enough. For hysteretic DQN and NoisyNet per episode, the size of the punishment barely matters: from -1 to -20, 4 or 5 runs out of 5 hunt. The temptation to forage matters more. With forage reward 3, foraging pays almost as much as hunting does for these agents; NoisyNet per episode still hunts in 5/5 runs, but hysteretic DQN only in 1/5 ($p = 0.048$). Plausibly, optimism alone cannot overcome a strong safe alternative, while consistent exploration brings the agents together before foraging becomes attractive. On larger grids, all runs of both methods still hunt, and catches fall roughly as the planner's do: learned agents reach roughly 40–60% of the planner's catches on every grid size.

3.6 A randomly moving stag requires active hunting

The "meet and wait" strategy relies on the stag walking into the agents. We removed this rule, so that the stag moves randomly (Figures 1b and 4). The task becomes much harder: even two scripted agents that always walk towards the stag make only 0.2 catches per 100 steps, so random exploration almost never produces a joint catch. With 300k training steps, no method learns to hunt. With hysteretic DQN and more training, active pursuit is learned: at 3M steps, 5/5 runs hunt with both separate networks (21.0 catches) and a shared network (23.3, slightly above the planner, whose one-step lookahead is not optimal for a random stag). The hunting agents behave very differently from the passive ones: they stay still in only 12% of the steps, but still chase the stag as one unit (no flanking). NoisyNet per episode, whose agents wait, never learns to hunt the random stag within 1M steps.

Hunting always appeared after ε had reached its minimum, which raises the question whether training length or exploration is the bottleneck. We tested this by ending exploration earlier (at 10% of a 1M-step run instead of 50%). With a shared network, this gives 5/5 hunting runs at 1M steps (22.3 catches, close to the 3M result), against 4/5 with the default schedule, and hunting appears earlier (after 530–760k steps instead of 790–980k). Ending exploration is not sufficient, though. Hunting never appears before about 0.5M steps, whatever the schedule, and the later ε ends, the shorter the wait after it, so experience collected while exploring also counts. Separate networks need about 2M steps (first hunting at 1.99–2.46M in the 3M runs) and do not hunt at 1M even with 900k low-ε steps. The answer is therefore *both*: active hunting needs low exploration and enough experience under it, and independent learners need more of it, as each must learn the value of chasing while its partner is still changing. Runs that have not yet learned to hunt show an intermediate stage, in which the agents already meet and move together (55–83% co-location) but still forage; learning proceeds from meeting, to moving together, to hunting.

4 Conclusion and limitations

In the Hunt game, independent DQN agents fail to cooperate because of how they *learn*: each agent's estimates are dragged down by its partner's exploration. An optimistic learning rule (hysteretic updates) or consistent exploration (per-episode NoisyNet) is enough for selfish agents with independent networks at the default rewards, and the resulting cooperation is robust to the size of the punishment and to the state size.

Limitations: (i) The default cooperation is passive and relies on the stag walking into the agents; active pursuit is learnable, but needs 3–10 times more training. (ii) The conventions do not transfer: agents trained together coordinate on arbitrary details, and fail badly with new partners. Training each agent with a population of different partners would be the natural next step. (iii) We use small numbers of seeds (3–5 per configuration); differences such as 5/5 against 4/5 are not significant. (iv) A seed fixes a run only on the same machine and library build: floating-point differences between platforms make single runs diverge, so our conclusions rest on counts over seeds rather than on individual runs.

5 Links to external resources

Code, run list and all trained runs: `https://github.com/marco-zocco/stag-hunt-with-RL`.
Environment: `https://github.com/giorgiofranceschelli/Gymnasium-Stag-Hunt`.

References

[1] Meire Fortunato, Mohammad Gheshlaghi Azar, Bilal Piot, Jacob Menick, Ian Osband, Alex Graves, Vlad Mnih, Remi Munos, Demis Hassabis, Olivier Pietquin, Charles Blundell, and Shane Legg. Noisy networks for exploration. In *International Conference on Learning Representations (ICLR)*, 2018.

[2] Laëtitia Matignon, Guillaume J. Laurent, and Nadine Le Fort-Piat. Hysteretic Q-learning: an algorithm for decentralized reinforcement learning in cooperative multi-agent teams. In *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 64–69, 2007.

[3] David Lvovich Nesterov-Rappoport. The evolution of trust: Understanding prosocial behavior in multi-agent reinforcement learning systems. Technical report, Drew University, Madison, NJ, 2022. Gym-Stag-Hunt environments; Gymnasium port by G. Franceschelli: `https://github.com/giorgiofranceschelli/Gymnasium-Stag-Hunt`.

[4] Shayegan Omidshafiei, Jason Pazis, Christopher Amato, Jonathan P. How, and John Vian. Deep decentralized multi-task multi-agent reinforcement learning under partial observability. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 2681–2690, 2017.

[5] Alexander Peysakhovich and Adam Lerer. Prosocial learning agents solve generalized stag hunts better than selfish ones. In *Proceedings of the 17th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, pages 2043–2044, 2018.

[6] Hado van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double Q-learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 2094–2100, 2016.